Project - Low Level Design

on

Construction Planning Assistant Agent

Course Name: Agentic-Ai

Institution Name: Medicaps University – Datagami Skill Based Course

Student Name(s) & Enrolment Number(s):

Sr no	Student Name	Enrolment Number
1.	Vivek Sharma	EN22ME304116
2.	Jayesh Chouhan	EN23MEEL4001
3.	Sakina Modi	EN22EL301044
4.	Vinay Birla	EN22CS3011083
5.	Dev Singh Ghuraiya	EN22ME304022

Group Name:05D10

Project Number:DO-18

Industry Mentor Name:

University Mentor Name: Maya yadav

Academic Year:2025-26

Table of Contents

1. Introduction.
   1.1. Scope of the document.
   1.2. Intended Audience
   1.3. System overview.

2. System Design.
   2.1. Application Design
   2.2. Process Flow.
   2.3. Information Flow.
   2.4. Components Design
   2.5. Key Design Considerations
   2.6. API Catalogue.

3. References

1. Introduction

1.1 Scope of the Document

This document provides a detailed low-level design for **Arch — Construction Planning Assistant**, an **AI agent-based system** that autonomously generates structured construction execution plans from natural language inputs.

Unlike traditional systems, Arch operates using an **LLM-driven agent loop**, where the system:

- Interprets user intent
- Plans execution steps
- Dynamically invokes tools
- Iteratively refines outputs

This document covers:

- Agent architecture and execution loop
- Tool-calling and reasoning workflow
- API interactions and streaming behavior
- Data flow and decision-making pipeline

The goal is to define how the **AI agent internally reasons, acts, and produces results** at an implementation level.

1.2 Intended Audience

This document is intended for:

- Backend Engineers working with LLM systems
- AI/ML Engineers building agentic workflows
- Full Stack Developers integrating AI APIs
- System Architects designing intelligent systems
- DevOps Engineers managing deployment
- Technical Reviewers evaluating system design

1.3 System Overview

Arch is an **LLM-powered autonomous agent system** that follows a **Reason → Act → Observe loop**.

The system:

- Accepts a construction goal in natural language
- Uses an LLM to **decompose the task into phases**
- Dynamically decides which tools to call
- Executes tool functions for real-world validation
- Streams intermediate reasoning and actions via SSE
- Produces a final structured execution plan

Agent Capabilities

- Task decomposition (planning)
- Tool selection (decision-making)
- External validation (execution)
- Iterative reasoning (feedback loop)

Core Technologies

- FastAPI (API + streaming backend)
- Groq (Llama 3.3 70B LLM)
- Server-Sent Events (real-time streaming)
- Vanilla JS (frontend interface)

2. System Design

2.1 Application Design

The system follows an **Agent-Oriented Layered Architecture**.

1. Presentation Layer

- Chat-based UI
- Shows agent reasoning + final output in real time

2. API Layer

- Built with FastAPI
- Handles SSE (streaming responses)
- Serves frontend SPA

3. Agent Execution Layer

- Core logic in agent_stream()
- Follows loop: **Reason** → **Act** → **Observe**
- *(Note: agents/ folder is legacy, not used)*

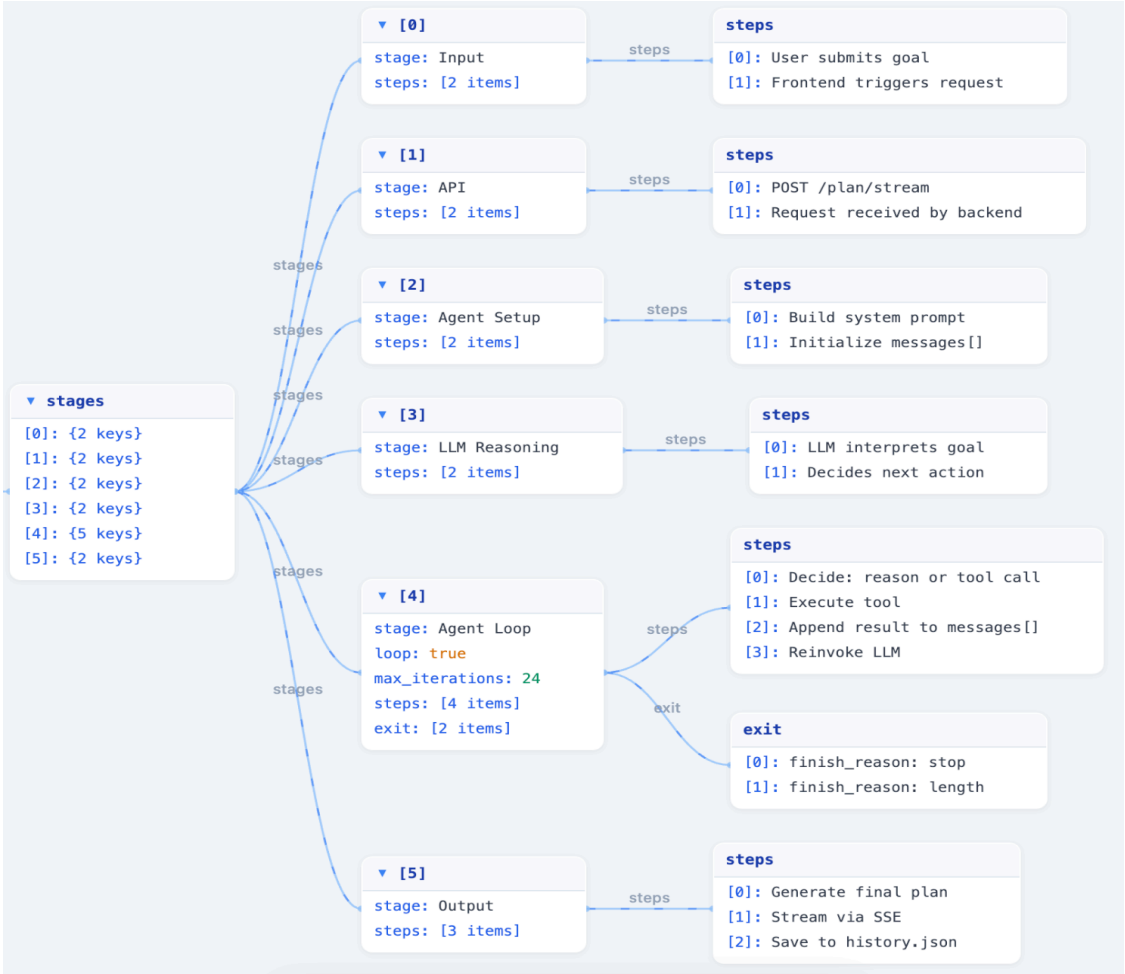
4. Tool Execution Layer

- Executes external function calls
- Currently simulated (random)
- Can be replaced with real APIs/DB

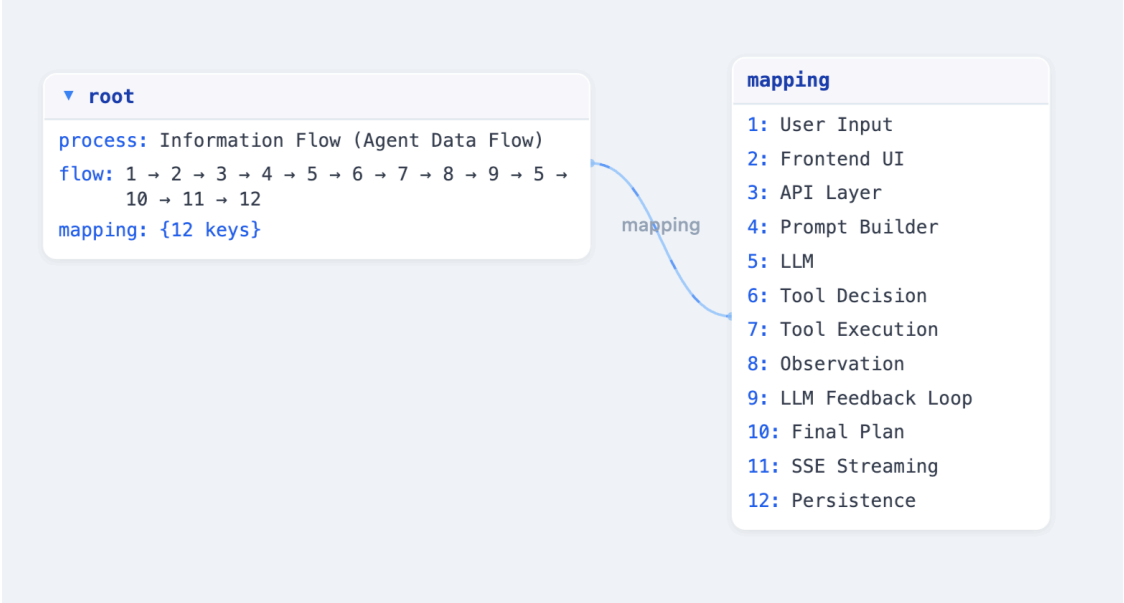
5. Persistence Layer

- Stores data in history.json
- Keeps last 100 plans

2.2 Process Flow (Agent Execution Flow)



2.3 Information Flow (Agent Data Flow)



Streaming Event Types

Event	Trigger	Payload
tool_call	Tool execution	name, arg, result
final	Final LLM response	content, id
error	Exception / rate limits	message

2.4 Components Design

1. Frontend

Handles user interaction.Takes input from the user and displays the generated plan, live streaming results, and history.

2. FastAPI Server

Acts as the backend layer. Manages API requests, handles SSE streaming, routes data, and connects all components.

3. AI Agent

Core brain of the system. Uses an LLM to understand the goal, perform reasoning, and execute tools in a loop to build the plan.

4. Tool Layer

Provides external support functions. Checks materials, workers, permits, and calculates duration for better decision-making.

5. Persistence Layer

Stores outputs. Saves generated plans and related data in `history.json` for future access.

2.5 Key Design Considerations

1. Scalability

- Stateless HTTP requests
- Stateful agent context (`messages[]`)
- Tool abstraction enables API replacement

2. Reliability

- Handles token truncation
- Retries tool failures
- Graceful rate-limit handling
- SSE-based error propagation

3. Performance

- Dynamic prompts reduce token usage
- Server-side tool execution
- SSE improves perceived latency

4. Extensibility

- Add tools via `TOOLS` and `TOOL_MAP`
- UI pill-based tool activation
- Prompt injection via `PILL_PROMPT_EXTRAS`

5. Maintainability

- Separation of concerns:
  - Reasoning (LLM)
  - Acting (tools)
  - State (`messages[]`)
  - Persistence (`history.json`)

2.6 API Catalogue

1. GET /

Serves the frontend (`index.html`)

2. POST /plan/stream

Triggers agent execution via SSE (Server-Sent Events)

Request :

```JSON
{
  "goal": "Plan a 3-story building",
  "required_tools": [
    "check_material_availability",
    "check_worker_availability",
    "check_permit_status",
    "calculate_duration"
  ]
}
```

Response (SSE):

```JSON
[
  {
    "type": "tool_call",
    "name": "check_material_availability",
    "arg": "Foundation",
    "result": "Concrete is delivered."
  },
  {
    "type": "final",
    "content": "## Project Overview...",
    "id": "uuid"
  },
  {
    "type": "error",
    "message": "Rate limit reached."
  }
]
```

3. GET /history

Returns stored outputs

4. DELETE /history/{id}

Deletes a specific record

Response

{"ok": true}

3. References:

- FastAPI Documentation – https://fastapi.tiangolo.com/
- Groq API Documentation – https://console.groq.com/docs
- Server-Sent Events (SSE) – https://developer.mozilla.org/en-US/docs/Web/API/Server-sent_events
- Llama 3.3 70B Model Card – https://huggingface.co/meta-llama